

Recursive Descendant Parser

Syntax Analysis

Ahmad Yoosofan

Compiler course

University of Kashan

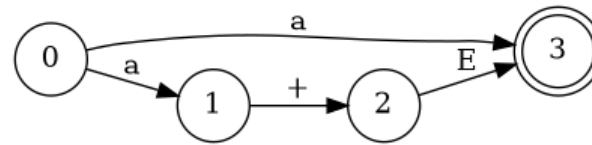
<https://yoosofan.github.io/course/compiler.html>

Special State Diagram for Addition

1. 2
2. 2+3
3. 86+54+876+432+32

Grammar

1. $E \rightarrow a$
 2. $E \rightarrow a + E$
- $E \rightarrow a + E \mid a$



Which edge must we choose from node 0?

Node 1

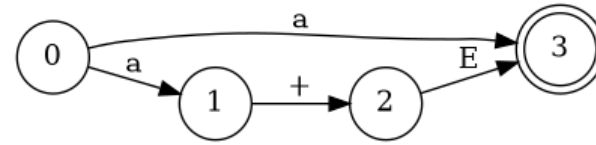
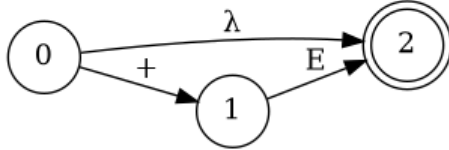
Node 3

Left Factoring

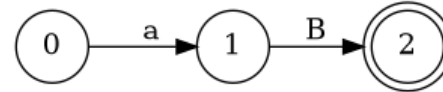
- $E \rightarrow a + E$
- $E \rightarrow a$

1. $E \rightarrow a B$
2. $B \rightarrow + E$
3. $B \rightarrow \lambda$

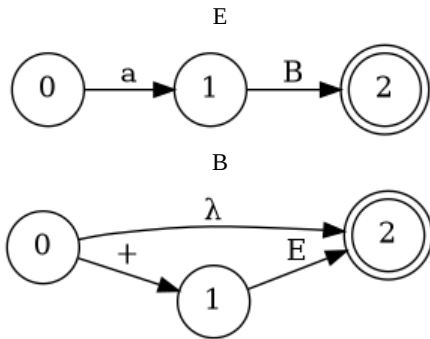
B



E



Parser Code for Add



```

1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Syntax Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))

```

```

python3 a.py 12.1 + 35.45 + 2
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
True

```

```

python3 a.py 12.1 + + 2
12.1 + + 2
a B a: 12.1, B: ++2
+E, E: +2
Syntax Error: number expected
Reminder: +2
False

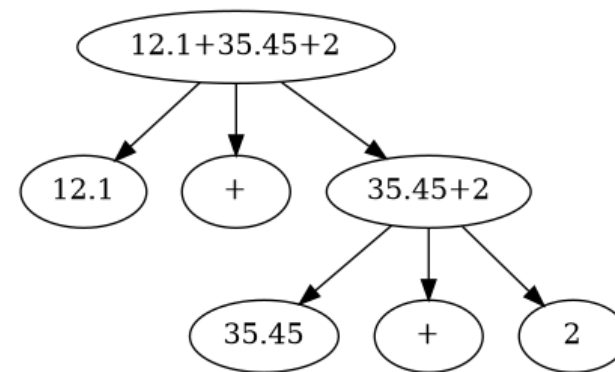
```

Parse Tree

```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Syntax Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))
```

```
python3 a.py 12.1 + 35.45 + 2
```

```
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
True
```



1. $E[12.1+35.45+2] \Rightarrow$
2. $a[12.1] + E[35.45+2] \Rightarrow$
3. $a[12.1] + a[35.45] + E[2] \Rightarrow$
4. $a[12.1] + a[35.45] + a[2] \Rightarrow$
5. Left Most Derivation

Doing Addition

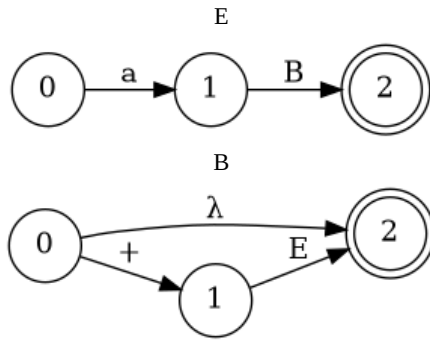
```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         return B(lex1)
7     print('Synatx Error: number expected')
8     print('Reminder: ', lex1.s[lex1.begin:])
9     return False
10
11 def B(lex1):
12     t = lex1.getToken()
13     if t.type == '+':
14         print('+E,\tE:', lex1.s[lex1.end:])
15         return E(lex1)
16     elif t.type == 'EOF': return True
17     print('Error a a:', s[0])
18     return False
19
20 def run(s):
21     lex1 = lexical_number_plus.lexical(s);
22     print(E(lex1))
```

```
1 def E(lex1):
2     t = lex1.getToken()
3     if t.type == 'a':
4         print('a B\t a: ', t.value, end=',')
5         print('\tB: ', lex1.s[lex1.end:])
6         res = B(lex1)
7         return res[0], t.value + res[1]
8     print('Synatx Error: number expected')
9     print('Reminder: ', lex1.s[lex1.begin:])
10    return False, 0
11
12 def B(lex1):
13     t = lex1.getToken()
14     if t.type == '+':
15         print('+E,\tE:', lex1.s[lex1.end:])
16         return E(lex1)
17     elif t.type == 'EOF':
18         return True, 0
19     print('Error a a:', s[0])
20    return False, 0
```

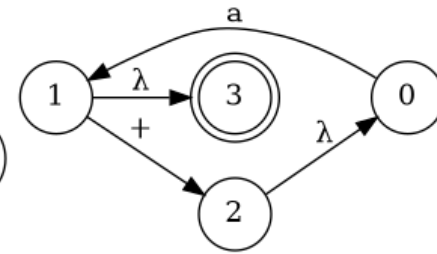
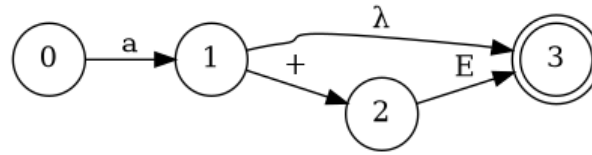
```
python3 a.py 12.1 + 35.45 + 2
a B a: 12.1, B: +35.45+2
+E, E: 35.45+2
a B a: 35.45, B: +2
+E, E: 2
a B a: 2.0, B:
(True, 49.550000000000004)
```

```
python3 a.py 2 +
a B a: 2.0, B: +
+E, E:
Synatx Error: number expected
Reminder:
(False, 2.0)
```

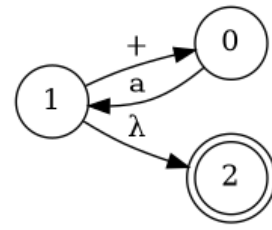
Simplify Diagram



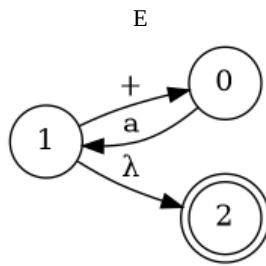
Combine them



E



Parser Code for add Nonrecursive

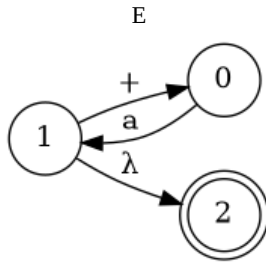


```
1 def E(lex1):
2     while True:
3         t = lex1.getToken()
4         if t.type == 'a':
5             print('a: ', t.value, end=',')
6             print('\t: ', lex1.s[lex1.end:])
7         else:
8             print('Synatx Error: number expected')
9             print('\t', lex1.s[lex1.begin:])
10            break
11        t = lex1.getToken()
12        if t.type != '+':
13            if t.type != 'EOF':
14                print('End of input expected')
15            break
16        if t.type != 'EOF': return False
17        return True
18
19 def run(s):
20     lex1 = lexical_number_plus.lexical(s);
21     print(E(lex1))
```

```
input: 16+34+0.30
output:
a: 16.0, : +34+0.30
a: 34.0, : +0.30
a: 0.3, :
True
```

```
input: 23+ + 34
output:
a: 23.0, : ++34
Synatx Error: number expected
+34
False
```

Calculator Code for add Nonrecursive

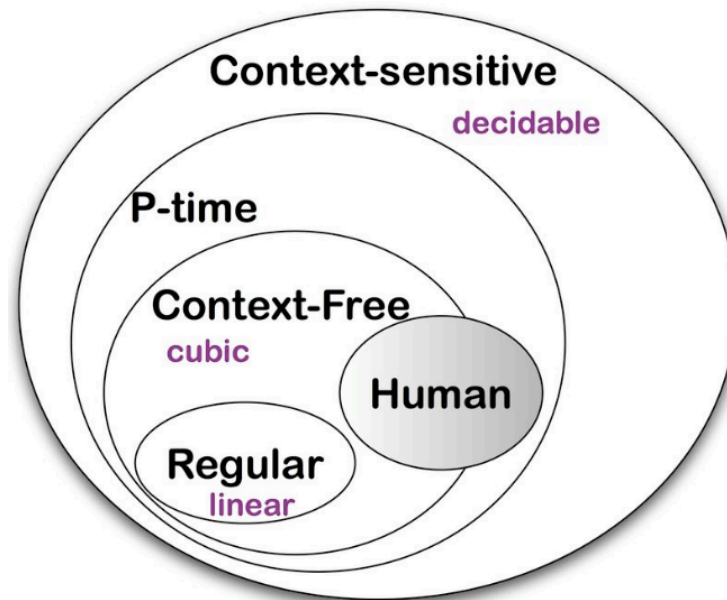


```
1 def E(lex1):
2     x = 0.0; state = 0; validity = True;
3     while True:
4         if state == 0:
5             t = lex1.getToken()
6             if t.type == 'a':
7                 print('a: ', t.value, end=',')
8                 print('\t: ', lex1.s[lex1.begin:]); validity = False; break
9                 state = 1; x += t.value;
10            else:
11                print('Synatx Error: number expected')
12                print('\t', lex1.s[lex1.begin:]); validity = False; break
13        elif state == 1:
14            t = lex1.getToken()
15            if t.type == '+': state = 0
16            elif t.type == 'EOF': state = 2
17            else: print('End of input expected'); validity = False; break;
18        elif state == 2: validity = True; break
19    return validity, x
```

```
input: 16+34+0.30
output:
a: 16.0, : +34+0.30
a: 34.0, : +0.30
a: 0.3, :
(True, 50.3)
```

```
input: 23+ + 34
output:
a: 23.0, : ++34
Synatx Error: number expected
+34
(False, 23.0)
```

Classes of Languages



https://www.researchgate.net/figure/Human-languages-and-the-classes-of-the-Chomsky-hierarchy-with-parsing-complexity_fig1_257299091

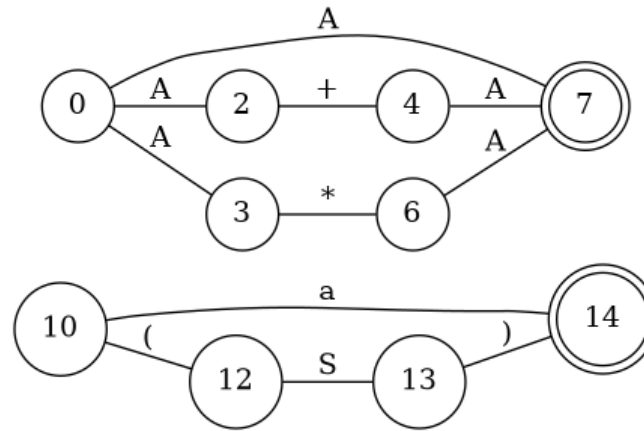
Parse for + * () no priority

Consider the following sentences of a language:

Write down the Grammar

1. 3
2. 9 + 3
3. 9 * 3
4. (9 * 3)
5. (9 + 3)
6. (9 * 4) + 5
7. 9 * (4 + 5)
8. (9 * (4 + 5)) * 3
9. 9 * ((4 + 5) * 3)
10. 9 * ((4 + 5) + 3)
11. (9 + (4 + 5)) * 3

1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow a$

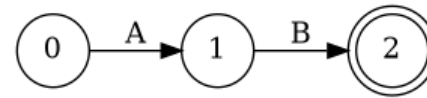


Left Factoring

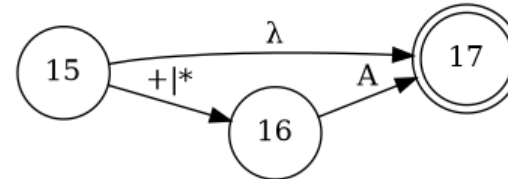
1. $S \rightarrow A + A$
2. $S \rightarrow A * A$
3. $S \rightarrow A$
4. $A \rightarrow (S)$
5. $A \rightarrow a$

1. $S \rightarrow A B$
2. $B \rightarrow +A \mid *A \mid \lambda$
3. $A \rightarrow (S) \mid a$

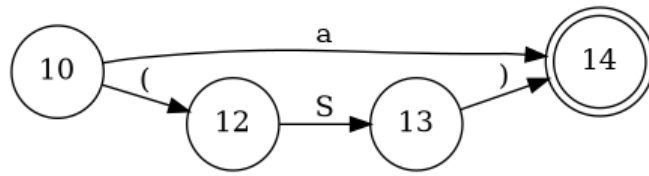
- S



- B

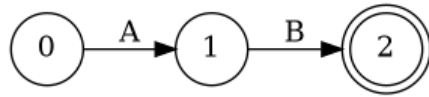


• A

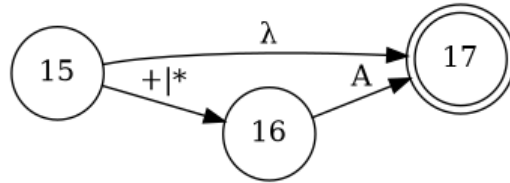


Simplify Diagram

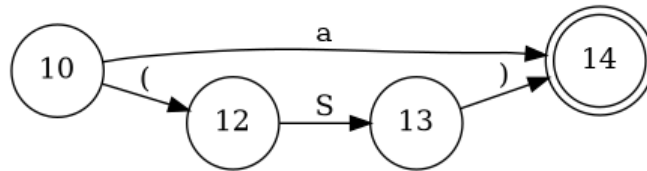
- S



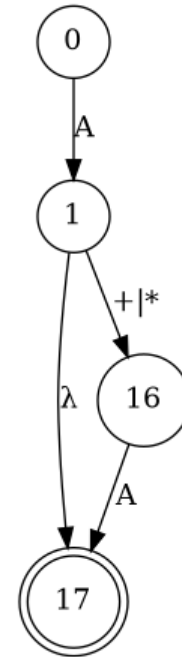
- B



- A

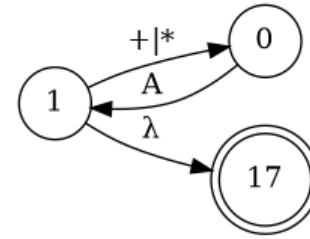
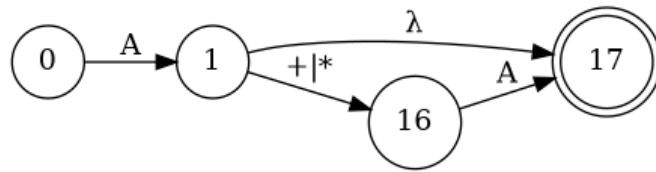


- Combine S and B

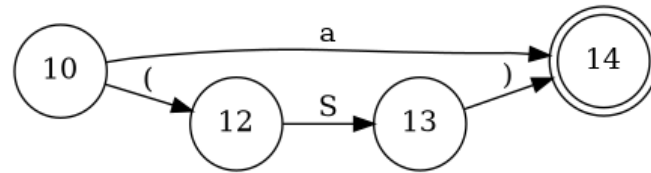


Remove Some Recursion

- S

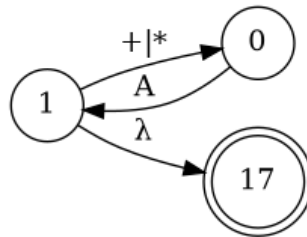


- A

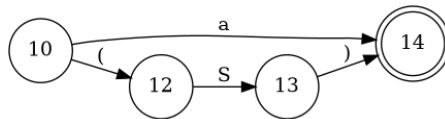


Parser Code

• S



• A



```

1 def S(lex1):
2     while True:
3         if A(lex1) == False: return False
4         if lex1.ct.type != '+' and lex1.ct.type != '*':
5             break
6         print(lex1.ct.type, ': ', lex1.s[lex1.end:])
7         lex1.getToken()
8         if lex1.ct.type == 'EOF' or lex1.ct.type == ')':
9             return True
10        return False
11 def A(lex1):
12     if lex1.ct.type == 'a':
13         print('a: ', lex1.ct.value, end=',')
14         print('\t: ', lex1.s[lex1.end:])
15         lex1.getToken()
16     elif lex1.ct.type == '(':
17         print('\t: ', lex1.s[lex1.end:])
18         lex1.getToken()
19         if S(lex1) == False: return False
20         if lex1.ct.type != ')':
21             print('Syntax Error: ) expected')
22             print(lex1.ct.type, '\t', lex1.s[lex1.begin:])
23             return False
24         else: print(''); lex1.getToken()
25     else:
26         print(' a or ( expected:: ', lex1.ct.type)
27         return False;
28     return True
29 def run(s):
30     lex1 = lexical_number_plus_mul_par.lexical(s);
31     lex1.getToken()
32     if S(lex1) == True:
  
```

```

1 import sys, lexical_number_plus_mul_par
2 def S(lex1):
3     while True:
4         if A(lex1) == False: return False
5         if lex1.ct.type != '+' and lex1.ct.type != '*':
6             break
7         print(lex1.ct.type, ':', lex1.s[lex1.end:])
8         lex1.getToken()
9         if lex1.ct.type == 'EOF' or lex1.ct.type == ')':
10            return True
11        return False
12 def A(lex1):
13     if lex1.ct.type == 'a':
14         print('a: ', lex1.ct.value, end=',')
15         print('\t: ', lex1.s[lex1.end:])
16         lex1.getToken()
17     elif lex1.ct.type == '(':
18         print('(\t: ', lex1.s[lex1.end:])
19         lex1.getToken()
20         if S(lex1) == False: return False
21         if lex1.ct.type != ')':
22             print('Syntax Error: ) expected')

```

```

21         print(lex1.ct.type, '\t', lex1.s[lex1.begin:
22             return False
23         else: print(')'); lex1.getToken()
24     else:
25         print(' a or ( expected:: ', lex1.ct.type)
26         return False;
27     return True
28 def run(s):
29     lex1 = lexical_number_plus_mul_par.lexical(s);
30     lex1.getToken()
31     if S(lex1) == True:
32         if lex1.ct.type == 'EOF':
33             print(True)
34         else:
35             print('Syntax Error: extra characters')
36             print(lex1.s[lex1.begin:])
37     else:
38         print('Syntax Error at')
39         print(lex1.s[lex1.begin:])
40 if __name__ == "__main__":
41     if len(sys.argv) < 2:
42         print("python3 a.py '12.1 + (35.45 * 2)'" )
43         raise SystemExit(f"Usage: {argvs[0]} input exp
44     s = ''.join(sys.argv[1:])
45     run(s)

```

Lexical

```
2 class tokenCls:
3     def __init__(self, type1='EOF',
4         v1='', l1=0, b1=0, e1=0):
5         self.type = type1
6         self.begin = b1
7         self.end = e1
8         self.value = v1
9         self.lineno = l1
10    def __str__(self):
11        return '[type: ' + str(self.type) + \
12            '\n\tvalue: ' + str(self.value) + \
13            '\n\tline: ' + str(self.lineno) + \
14            '\n\tbegin: ' + str(self.begin) + \
15            '\n\tend: ' + str(self.end) + ']'
16 class lexical:
17     def __init__(self, s):
18         self.s = s
19         self.begin = 0
20         self.end = 0
21         self.lineno = 0
22         self.length = len(self.s)
23         self.ct = tokenCls(b1=self.begin,
24             e1=self.end)
25     def getToken(self):
26         self.ct = tokenCls(b1=self.begin,
27             e1=self.end)
28         self.begin = self.end
29         while self.end < self.length:
30             a = re.match('[ \t]+', self.s[self.end:])
31             if a is not None:
32                 self.end = self.begin = self.end + a.end()
33             if self.end >= self.length:
```

```
33         break
34
35         a = re.match('\d+(\.\d+)?',
36             self.s[self.end:])
37         if a is None:
38             self.end += 1
39         operators1 = ['+', '-', '*', '/', '(', ')']
40         if self.s[self.begin:self.end] in operators:
41             self.ct.type = self.s[self.begin:self.end]
42             break
43         else:
44             print('Lexical error: ', self.lineno, end=
45                 '\n', self.begin, ': ', self.end, end=
46                 '\n', self.s[self.begin:self.end],
47                 '\nUnknown character: ', self.s[self.b
48                 self.begin += 1
49         else:
50             self.ct.type = 'a'
51             self.ct.value = float(a.group())
52             self.end += a.end()
53             break
54         self.ct.begin = self.begin
55         self.ct.end = self.end
56         self.ct.lineno = self.lineno
57         return self.ct
58 if __name__ == '__main__':
59     #s = input('Enter ') # 12+35+2
60     s = ' 12 + 35 + 2 '
61     s = ' 12 + 35 --- 2 '
62     # ~ s = '1.0.' s = '1.0=23.2+4'
63     s = '12+35+2 '
64     lex1 = lexical(s);
```

Error Recovery (panic mode)

A Simple Calculator (,), +, *

1. 34
2. $34 + 2$
3. $34 + 45 + 98$
4. $34 + 45 + 98 * 4 * 554$
5. $(34 * 2) + 3$
6. $43 * (54 + 3)$
7. $2 + (34)$
8. $(34 + 3) * 2$
9. $34 + 3 * 2 * ((4))$

Grammar 1

1. $S \rightarrow S + S$
2. $S \rightarrow S * S$
3. $S \rightarrow a$
4. $S \rightarrow (S)$

Wrong Grammar (Why?)

Grammar 2

1. $E \rightarrow T + E \mid T$
2. $T \rightarrow F * T \mid F$
3. $F \rightarrow (E) \mid a$

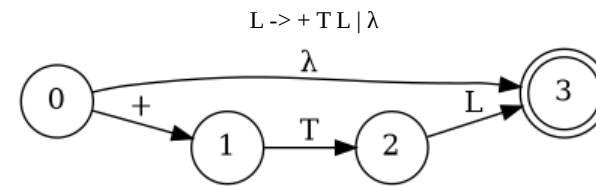
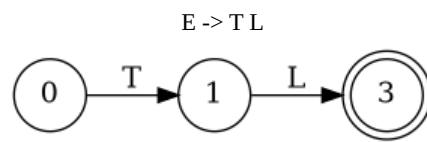
Grammar 3

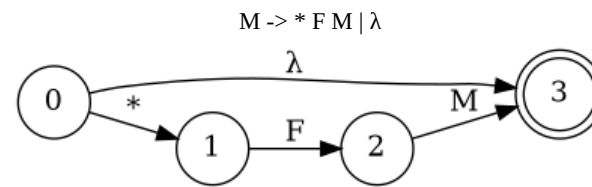
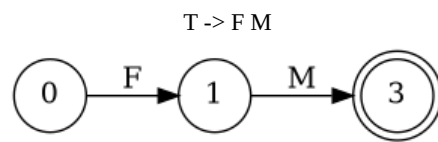
- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid a$

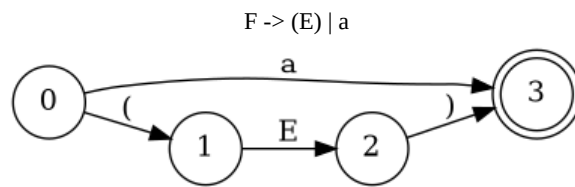
Left Recursion

1. $E \rightarrow T L$
2. $L \rightarrow + T L \mid \lambda$
3. $T \rightarrow F M$
4. $M \rightarrow * F M \mid \lambda$
5. $F \rightarrow (E) \mid a$

State Diagram(II)

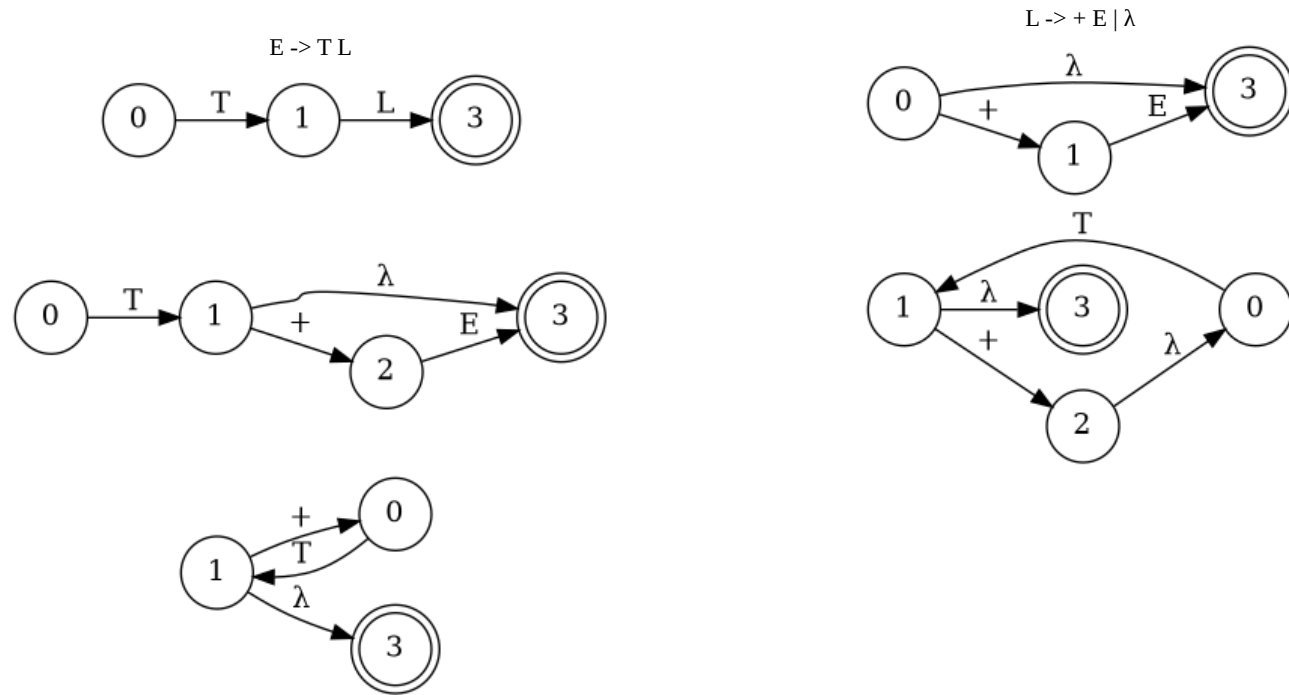






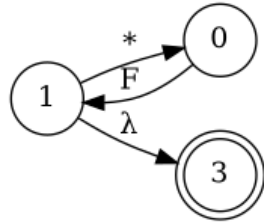
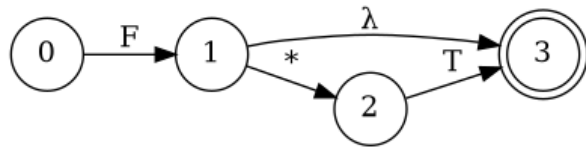
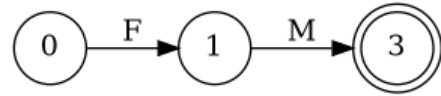
1. $E \rightarrow T L$
2. $L \rightarrow + E \mid \lambda$
3. $T \rightarrow F M$
4. $M \rightarrow * T \mid \lambda$
5. $F \rightarrow (E) \mid a$

Simplify Diagram(II) - Combine E, L

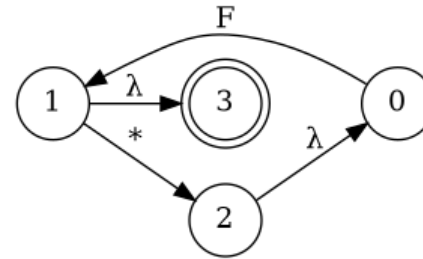
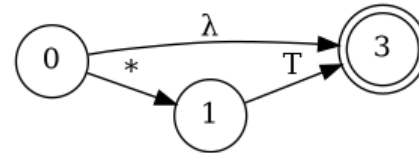


Simplify Diagram(III) - Combine T, M

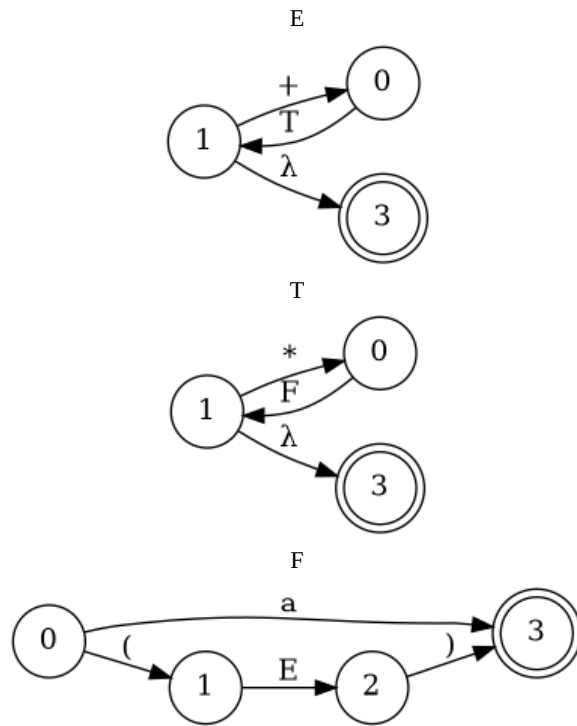
$T \rightarrow FM$



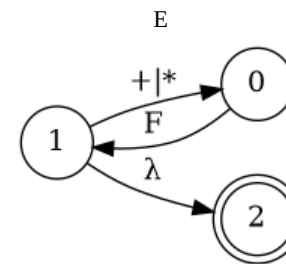
$M \rightarrow *T | \lambda$



Diagrams of the Second Calculator



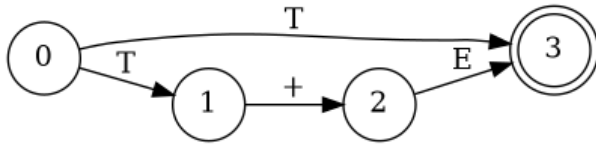
Can E and T be combined?



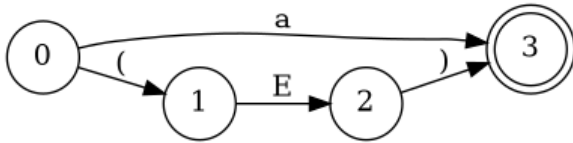
1. $4 + 3 * 2$
2. $(4 + 3) * 2$
3. $4 + (3 * 2)$
4. $3 * 2 + 4$
5. $3 * (2 + 4)$
6. $(3 * 2) + 4$

State Diagram

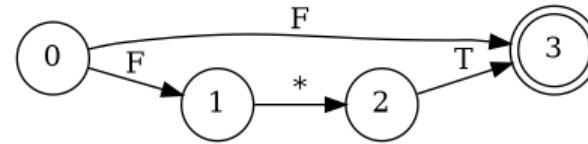
- $E \rightarrow T + E \mid T$



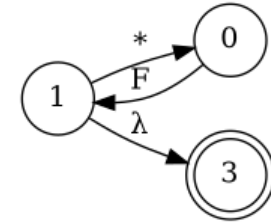
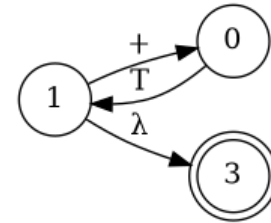
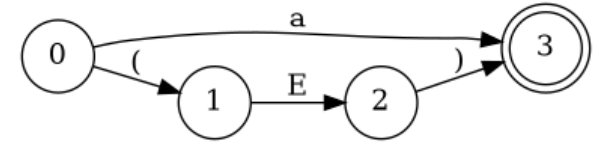
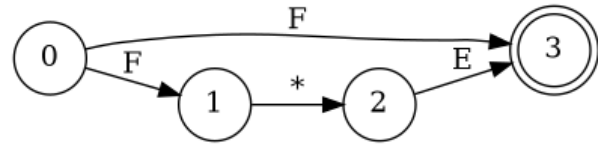
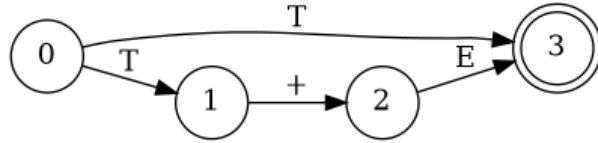
- $F \rightarrow (E) \mid a$



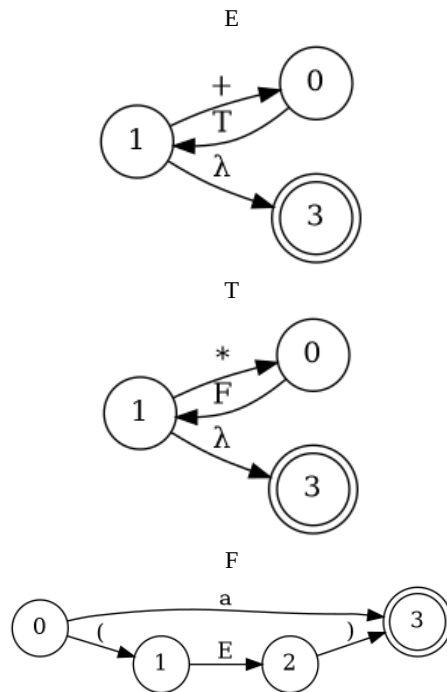
- $T \rightarrow F * T \mid F$



Simplify Diagram(IV) - Fewer steps



Parser Code for Add and Multiply



```

1 def S(lex1):
2     lex1.getToken()
3     if E(lex1) == True:
4         if lex1.ct.type == 'EOF': print(True)
5     else:
6         print('Syntax Error: extra characters at END: ')
7         print(lex1.s[lex1.begin:])
8     else: print('Syntax Error at', lex1.s[lex1.begin:])
9 def E(lex1):
10    while True:
11        if T(lex1) == False: return False
12        if lex1.ct.type != '+': break
13        lex1.getToken()
14    return True
15 def T(lex1):
16    while True:
17        if F(lex1) == False: return False
18        if lex1.ct.type != '*': break
19        lex1.getToken()
20    return True;
21 def F(lex1):
22     if lex1.ct.type == 'a':
23         lex1.getToken()
24     elif lex1.ct.type == '(':
25         lex1.getToken()
26         if E(lex1) == False: return False
27         if lex1.ct.type != ')': return False
28         lex1.getToken()
29     else: return False
30    return True
31 def run(s):
32     lex1 = lexical_number_plus_mul_par.lexical(s);
33     S(lex1)

```

Another Language for simple expression $+ - * / ()$

1. $2 + 3 * 4 - 5$
2. $4 - 5 * 2 / (4 - 2) + 1$
3. $((2 * (3 - 1)) / (5 - 3)) * (7 - 8)$

Grammar ?

1. $E \rightarrow T + E \mid T - E \mid T$
2. $T \rightarrow F * T \mid F / T \mid F$
3. $F \rightarrow (E) \mid a$

Is there any problem?

1. $4 - 3 - 2$
2. $16 / 4 / 2$

It was a wrong grammar

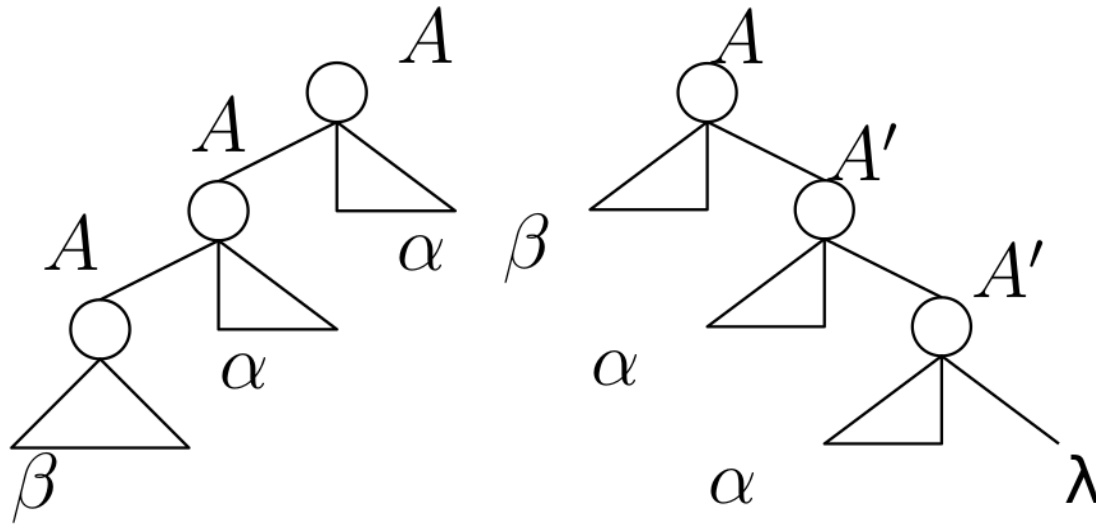
1. $E \rightarrow E + T \mid E - T \mid T$
2. $T \rightarrow T * F \mid T / F \mid F$
3. $F \rightarrow (E) \mid a$

Left Most Derivation

1. $E \Rightarrow E + T \Rightarrow$
2. $T + T \Rightarrow$
3. $F + T \Rightarrow$
4. $16 + T \Rightarrow 16 + T * F \Rightarrow$
5. $16 + F * F \Rightarrow$
6. $16 + 3 * F \Rightarrow$
7. $16 + 3 * 4$

Left Recursion Elimination

$$A \rightarrow A \alpha \mid \beta$$



- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \lambda$

General Form of Direct Left Recursion Elimination

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

Convert to

- $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$
- $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \lambda$

Calculator Grammar

$$A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_m | \beta_1 | \beta_2 | \dots | \beta_n$$

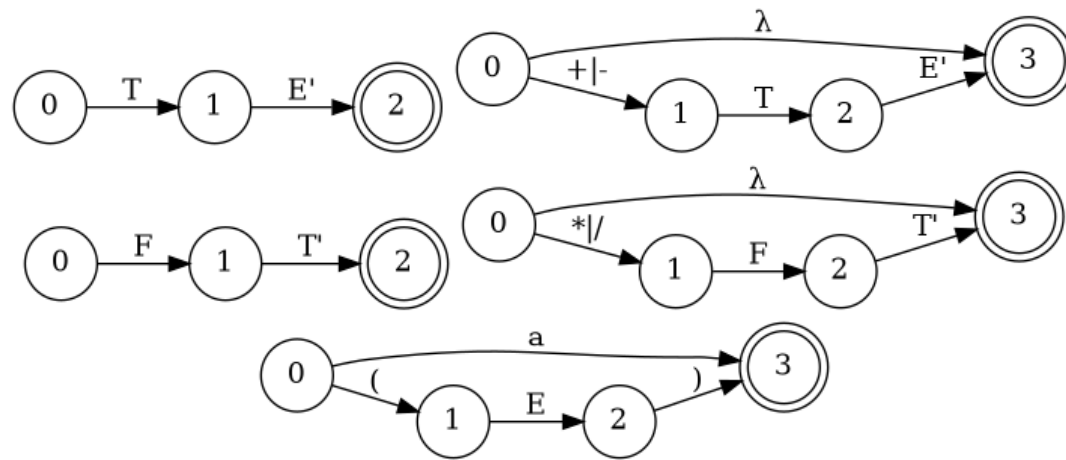
Convert to

- $A \rightarrow \beta_1 A' | \beta_2 A' | \dots | \beta_n A'$
- $A' \rightarrow \alpha_1 A' | \alpha_2 A' | \dots | \alpha_m A' | \lambda$

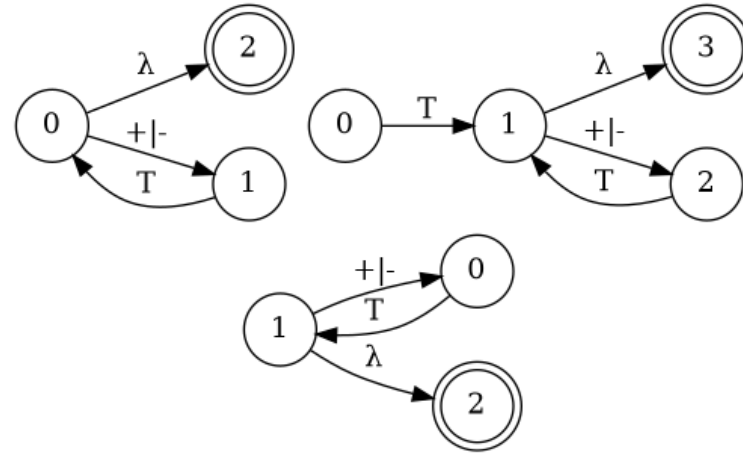
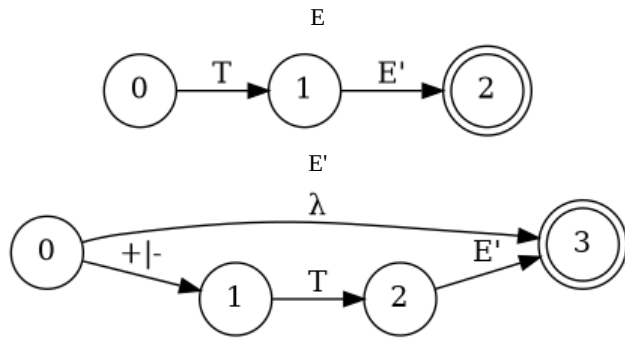
- $E \rightarrow E + T | E - T | T$
- $T \rightarrow T * F | T / F | F$
- $F \rightarrow a | (E)$

Convert to

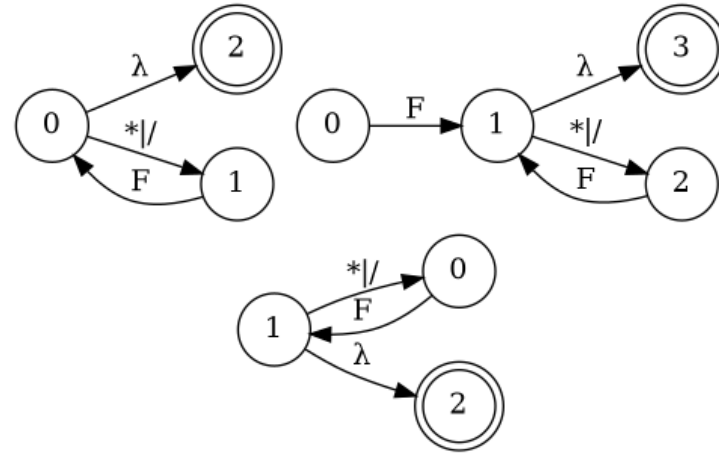
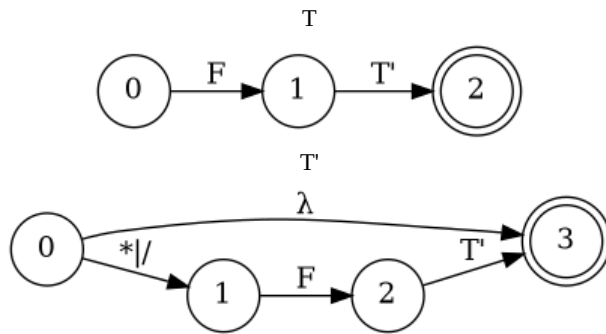
- $E \rightarrow T E'$
- $E' \rightarrow + T E' | - T E' | \lambda$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' | / F T' | \lambda$
- $F \rightarrow a | (E)$



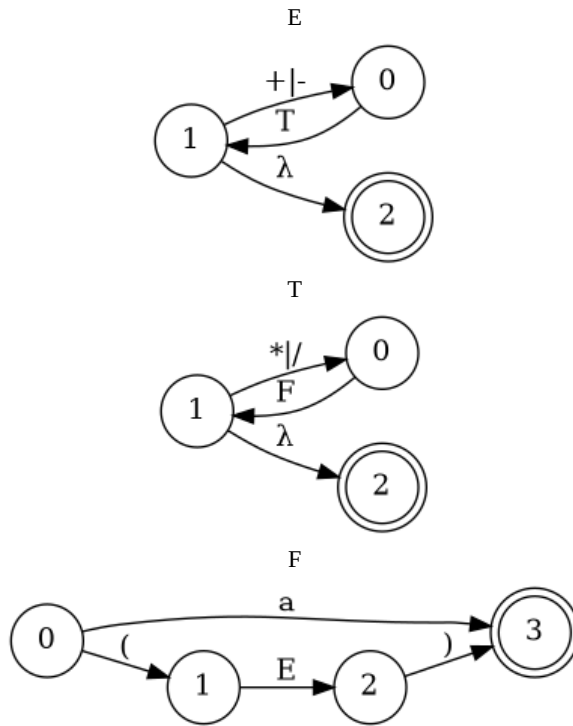
Simplify Diagram(V) - Combine E and E'



Simplify Diagram(VI) - Combining T and T'



Parser Code for Last Calculator



```

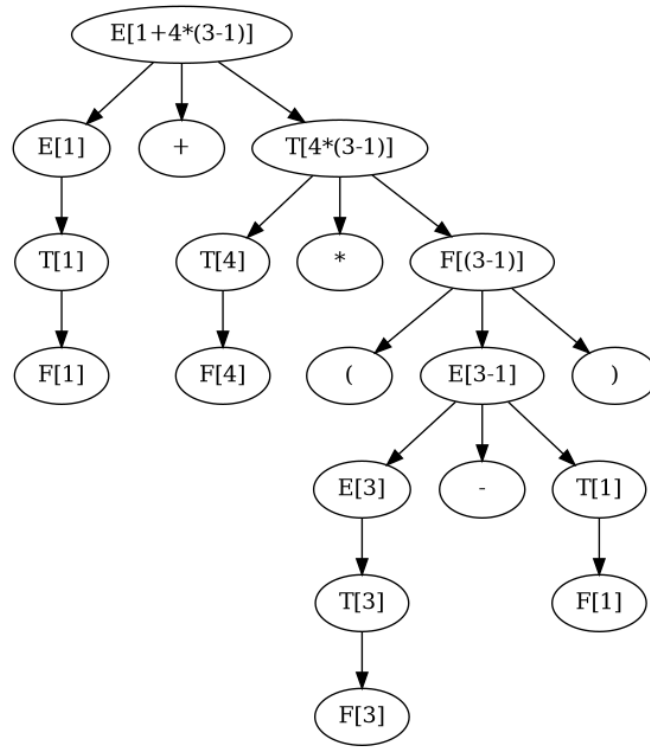
1 def S(lex1):
2     lex1.getToken()
3     if E(lex1) == True:
4         if lex1.ct.type == 'EOF': print(True)
5     else:
6         print('Syntax Error: extra characters at END: ')
7         print(lex1.s[lex1.begin:])
8     else: print('Syntax Error at', lex1.s[lex1.begin:])
9 def E(lex1):
10    while True:
11        if T(lex1) == False: return False
12        if lex1.ct.type not in ['+', '-']: break
13        lex1.getToken()
14    return True
15 def T(lex1):
16    while True:
17        if F(lex1) == False: return False
18        if lex1.ct.type not in ['*', '/']: break
19        lex1.getToken()
20    return True;
21 def F(lex1):
22     if lex1.ct.type == 'a':
23         lex1.getToken()
24     elif lex1.ct.type == '(':
25         lex1.getToken()
26         if E(lex1) == False: return False
27         if lex1.ct.type != ')': return False
28         lex1.getToken()
29     else: return False
30    return True
31 def run(s):
32     lex1 = lexical_number_plus_mul_par.lexical(s);
33     S(lex1)

```

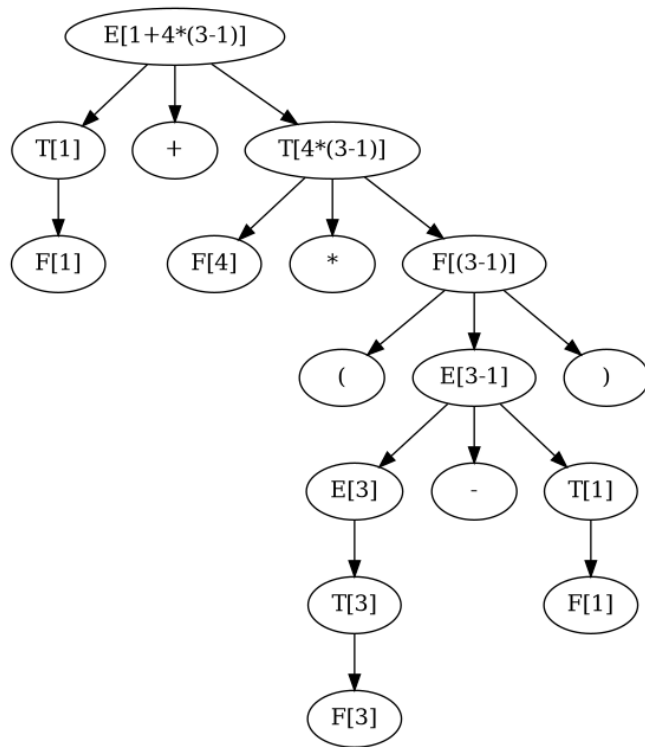
Parser Tree

```
9 def E(lex1):
10     while True:
11         print('E: ', lex1.s[lex1.begin:])
12         if T(lex1) == False: return False
13         if lex1.ct.type not in ['+', '-']:
14             break
15         print('Op: ', lex1.s[lex1.begin:])
16         lex1.getToken()
17     return True
18 def T(lex1):
19     while True:
20         print('T: ', lex1.s[lex1.begin:])
21         if F(lex1) == False: return False
22         if lex1.ct.type not in ['*', '/']:
23             break
24         print('Op: ', lex1.s[lex1.begin:])
25         lex1.getToken()
26     return True;
27 def F(lex1):
28     print('F: ', lex1.s[lex1.begin:])
29     if lex1.ct.type == 'a':
30         print('Number: ', lex1.ct.value)
31         lex1.getToken()
32     elif lex1.ct.type == '(':
33         lex1.getToken()
34         if E(lex1) == False: return False
35         if lex1.ct.type != ')':
36             return False
37         lex1.getToken()
38     else: return False
39     return True
```

```
1 python3 t.py '1+4*(3-1)'
2 E:  1+4*(3-1)
3 T:  1+4*(3-1)
4 F:  1+4*(3-1)
5 Number:  1.0
6 Op:  +4*(3-1)
7 T:  4*(3-1)
8 F:  4*(3-1)
9 Number:  4.0
10 Op:  *(3-1)
11 F:  (3-1)
12 E:  3-1)
13 T:  3-1)
14 F:  3-1)
15 Number:  3.0
16 Op:  -1)
17 T:  1)
18 F:  1)
19 Number:  1.0
20 True
```



1. $E \rightarrow E + T \mid E - T \mid T$, 2. $T \rightarrow T * F \mid T / F \mid F$, 3. $F \rightarrow (E) \mid a$



```

1 python3 t.py '1+4*(3-1)'
2 E: 1+4*(3-1)
3 T: 1+4*(3-1)
4 F: 1+4*(3-1)
5 Number: 1.0
6 Op: +4*(3-1)
7 T: 4*(3-1)
8 F: 4*(3-1)
9 Number: 4.0
10 Op: *(3-1)
11 F: (3-1)
12 E: 3-1)
13 T: 3-1)
14 F: 3-1)
15 Number: 3.0
16 Op: -1)
17 T: 1)
18 F: 1)
19 Number: 1.0
20 True

```


Calculator

```
2 def E(s1):
3     [s1, x]=T(s1)
4     while len(s1) > 0 and \
5         (s1[0] == '+' or s1[0] == '-'):
6         if s1[0]=='+' :
7             flag = True
8         else:
9             flag = False;
10        [s1, y] = T(s1[1:])
11        if flag == True:
12            x += y;
13        else:
14            x -= y
15    return [s1, x]
16
17 def T(s1):
18     [s1, x] = F(s1)
19     while len(s1) > 0 and \
20         (s1[0] == '*' or s1[0] == '/'):
21         if s1[0] == '*':
22             flag = True;
23         else:
24             flag = False;
25        [s1, y] = F(s1[1:])
26        if flag == True:
27            x *= y;
28        else:
29            x /= y
30    return [s1, x]
```

```
31 def F(s1):
32     if s1[0] == '(':
33         s1 = s1[1:]
34         [s1, x] = E(s1)
35         if len(s1) == 0 or s1[0] != ')':
36             print('Error: ) missing')
37             raise
38         s1 = s1[1:]
39         return [s1, x]
40     else:
41         a = re.match('[0-9]+(\\.[0-9]+)?', s1)
42         s1 = s1[a.end():]
43         return [s1, float(a.group())]
44
45 st = input('Enter >> ') #st='12+35-2'
46 while st != "":
47     try:
48         [s1, val] = E(st)
49         print(val)
50     except:
51         print('Error')
52     st = input('Enter >> ') #st='12+35-2'
```

Calculator(IV)(lexical)

```
1 import re
2 class error(BaseException):
3     def __init__(self,s):
4         self.s = s;
5
6 class Lexical:
7     def __init__(self, s = '4+5'):
8         self.ct = ['$ ', 0] # current token
9         self.s = s
10        self.index = 0
11        self.pi = 0
12        print(self.s);
13
14    def getToken(self):
15        res = []
16        self.pi = self.index;
17        if len(self.s) == self.index:
18            res=['$', 0]
19        elif self.s[self.index] == '*':
20            res=['*', 0]
21        elif self.s[self.index] == '/':
22            res=['/', 0]
23        elif self.s[self.index] == '+':
24            res=['+', 0]
25        elif self.s[self.index] == '-':
```

```
25         res=['-', 0]
26        elif self.s[self.index] == '(':
27            res=['(', 0]
28        elif self.s[self.index] == ')':
29            res=[')',0]
30        elif '0' <= self.s[self.index] <= '9':
31            a = re.match('[0-9]+(\.[0-9]+)?',
32                self.s[self.index:])
33            res = ['n', float(a.group())]
34            self.index += a.end() - 1;
35        else: raise error('lexical error ')
36        self.index += 1
37        self.ct = res # print(self.ct)
38        return res
39
40    def unget(self):
41        self.index = self.pi;
42
43    def E(lex):
44        x = T(lex);
45        while lex.ct[0] in ['+', '-']:
46            p = lex.ct[0]
```

Calculator(V)

```
47     lex.getToken()
48     y = T(lex);
49     if p == '+': x += y
50     else: x -= y;
51 return x;
52
53 def T(lex):
54     x = F(lex);
55     while lex.ct[0] in ['*', '/']:
56         p=lex.ct[0]
57         lex.getToken()
58         y = F(lex);
59         if p == '*': x *= y
60         else:      x /= y;
61 return x;
62
63 def F(lex):
64     x = 0
65     if lex.ct[0] == 'n':
66         x = lex.ct[1];
67     elif lex.ct[0] == '(':
68         lex.getToken()
69         x = E(lex);
```

```
70     if lex.ct[0] != ')' or lex.ct[0] == '$':
71         raise error(' ) is missing ')
72     else:
73         raise error(' Error in F ');
74     lex.getToken()
75     return x;
76
77 s = input('Enter >> ');
78 while s != '':
79     try:
80         lex = Lexical(s)
81         lex.getToken()
82         print(E(lex));
83     except error: print(' ) is missing');
84     except: print('Syntax Error')
85     s = input('Enter >> ')
```

Calculator(VI) - lexical

```
1  #include<iostream>
2  #include<cstdlib>
3
4  enum TokenType{
5      NUMBER, PLUS, MINUS, MUL, DIV,
6      LPAR, RPAR, END_OF_INPUT
7  };
8  struct Token
9  {TokenType type; int n; double r;};
10
11 void error(const std::string msg){
12     std::cout << msg << std::endl;
13     exit(0);
14 }
15
16 Token t;
17 char buffer[200];
18 int tindex = 0;
19
20 void E(void);
21 void T(void);
22 void F(void);
23
24 void read(void){
25     std::cout << "Enter expression"
26         << std::endl;
27     std::cin>>buffer;
28 }
29
30 Token getToken(void){
31     static int number_of_call = 0;
```

```
31     number_of_call ++;
32     if(buffer[tindex] == '(')
33         t.type = LPAR;
34     else if(buffer[tindex] == ')')
35         t.type = RPAR;
36     else if(buffer[tindex] == '+')
37         t.type = PLUS;
38     else if(buffer[tindex] == '-')
39         t.type = MINUS;
40     else if(buffer[tindex] == '*')
41         t.type = MUL;
42     else if(buffer[tindex] == '/')
43         t.type = DIV;
44     else if(buffer[tindex] >= '0' &&
45         buffer[tindex]<='9'){
46         t.type=NUMBER;
47         t.n = 0;
48         for(; buffer[tindex] >= '0'
49             && buffer[tindex] <='9'; tindex++)
50             t.n = t.n * 10 + buffer[tindex] - 48;
51         tindex --;
52     }
53     else if(buffer[tindex] == '\\0')
54         t.type = END_OF_INPUT;
55     else
56         error("unknown character");
57     //std::cout << t.type<<'\\t'<<tindex<<'\\t'
58     //<<number_of_call<<'\\t'<<t.n<<std::endl;
59     tindex++;
60     return t;
61 }
```

Calculator(VII)

```
63 void A(void){
64     t = getToken();
65     E();
66     if(t.type != END_OF_INPUT)
67         error("unrecognized token");
68     else
69         std::cout << "Accept" << std::endl;
70 }
71
72 void E(void){
73     T();
74     while(t.type == PLUS || t.type == MINUS){
75         t = getToken();
76         T();
77     }
78 }
79
80 void T(void){
81     F();
82     while(t.type == MUL || t.type == DIV ){
83         t = getToken();
84         F();
85     }
86 }
87
88 void F(void){
89     if(t.type == LPAR){
```

```
90         t = getToken();
91         E();
92         if(t.type != RPAR)
93             error(" ) is missing");
94     }
95     else if(t.type==NUMBER)
96         ;
97     else
98         error("Wrong token ");
99     t = getToken();
100 }
101
102 void tryGetToken(void){
103     do{
104         t = getToken();
105     }while(t.type != END_OF_INPUT);}
106
107 int main(){
108     read(); /*tryGetToken();*/
109     A();
110     return 0;
111 }/*2-(3-2)/(3-(2-1)/(5-2*2))-1+2*/
```

End

